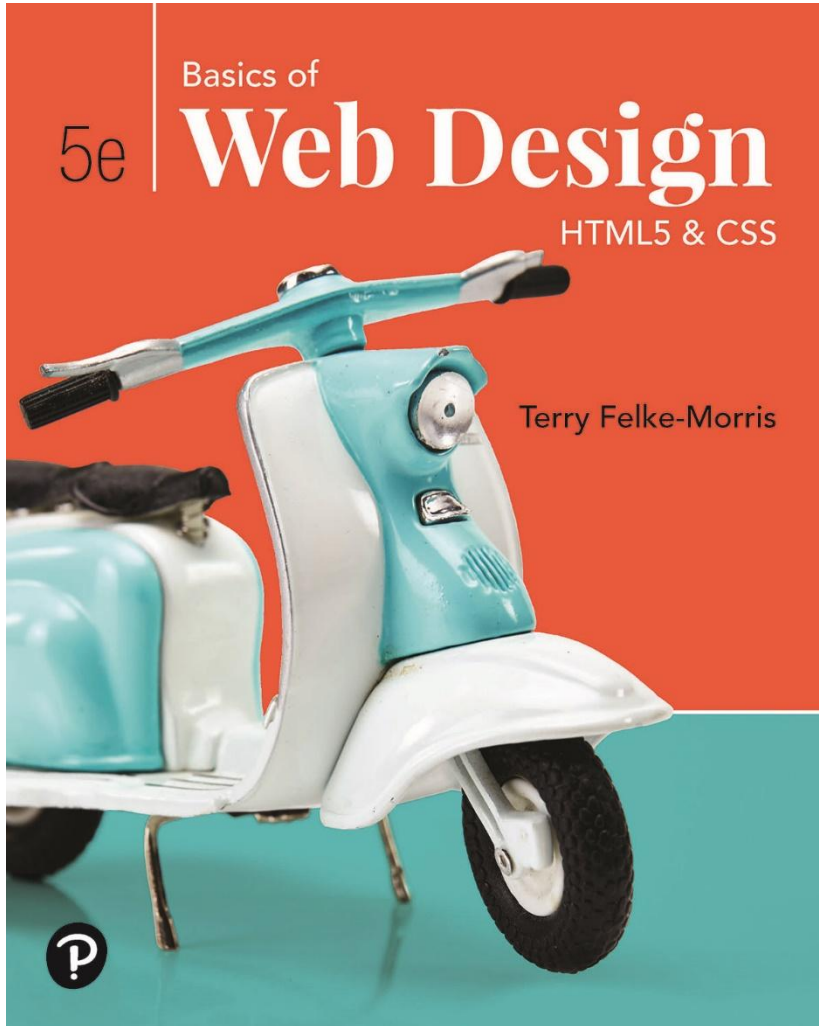


Basics of Web Design

Fifth Edition



Chapter 8

Responsive Layout Basics
Key Concepts

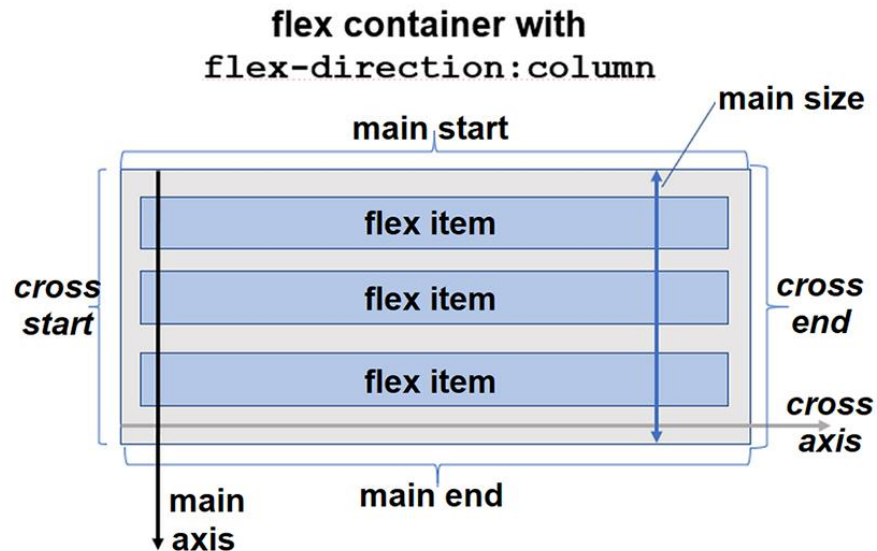
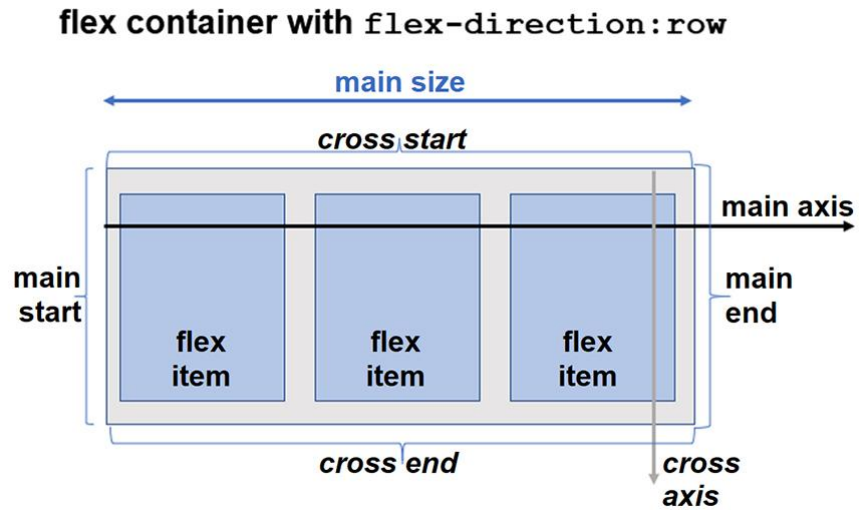
Learning Outcomes

- Describe the purpose of CSS Flexible Box Layout
- Configure a web page that applies CSS Flexible Box Layout
- Configure web pages for mobile display using the viewport meta tag
- Apply responsive image techniques including the new HTML5 picture element

CSS Flexible Box Layout aka flexbox

- Purpose: Provide for a flexible, responsive layout
<https://www.w3.org/TR/css-flexbox-1/>
- Best used for one dimension – a row or a column
- The display property
 - Configures a flexbox container`display: flex;`
- Flex Item -- a child element of the flex container
- The flex-wrap property
 - Determines whether flex items are displayed on multiple lines
 - Values are nowrap (default), wrap, wrap-reverse
- The flex-direction property
 - Configures the flow direction
 - Values are row,(default), column, row-reverse, and column-reverse

Diagram of a Flex Container



CSS Flexible Box Layout Module

CSS Flexible Box Layout Module

Before the **Flexible Box Layout module**, there were **four** layout modes:

- Block, for sections in a webpage
- Inline, for text
- Table, for two-dimensional table data
- Positioned, for explicit position of an element

Why Flexbox?

- A lot of flexibility
- Arrange items
- Spacing
- Alignment
- Order of items

CSS Flexbox Components

❑ A flexbox always consists of:

a **Flex Container** - the parent (container) `<div>` element

Flex Items - the items inside the container `<div>`

A Flex Container with Three Flex Items

To start using CSS Flexbox, you need to first define a flex container.
The flex container becomes flexible by setting the display property to flex.

Example

A flex container with three flex items:

```
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>
```

```
<style>
.flex-container {
  display: flex;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  margin: 10px;
  padding: 20px;
  font-size: 30px;
}
</style>
```


The CSS Flex Container

Like we specified in the previous chapter, this is a flex container (the blue area) with three flex items:

The flex container becomes flexible by setting the display property to flex:

Example

```
.flex-container {  
  display: flex;  
}
```

The CSS properties we use for the flex container are:

- flex-direction
- flex-wrap
- flex-flow
- justify-content
- align-items
- align-content

The CSS flex-direction Property

The `flex-direction` property specifies the display-direction of flex items in the flex container.

The `flex-direction` property can have one of the following values:

- `row`
- `column`
- `row-reverse`
- `column-reverse`

Example

The `row` value is the default value, and it displays the flex items horizontally (from left to right):

```
.flex-container {  
  display: flex;  
  flex-direction: row;  
}
```

Example

The **column** value displays the flex items vertically (from top to bottom):

```
.flex-container {  
  display: flex;  
  flex-direction: column;  
}
```

Example

The **row-reverse** value displays the flex items horizontally (but from right to left):

```
.flex-container { display: flex;  
  flex-direction: row-reverse;  
}
```

Example

The `column-reverse` value displays the flex items vertically (but from bottom to top):

```
.flex-container {  
  display: flex;  
  flex-direction: column-reverse;  
}
```

The CSS flex-wrap Property

The `flex-wrap` property specifies whether the flex items should wrap or not, if there is not enough room for them on one flex line.

The `flex-wrap` property can have one of the following values:

- `nowrap`
- `wrap`
- `wrap-reverse`

Example

The `nowrap` value specifies that the flex items will not wrap (this is default):

```
.flex-container {  
  display: flex;  
  flex-wrap: nowrap;  
}
```

Example

The **wrap** value specifies that the flex items will wrap if necessary:

```
.flex-container {  
  display: flex;  
  flex-wrap: wrap;  
}
```

Example

The **wrap-reverse** value specifies that the flex items will wrap if necessary, in reverse order:

```
.flex-container {  
  display: flex;  
  flex-wrap: wrap-reverse;  
}
```

The CSS flex-flow Property

The `flex-flow` property is a shorthand property for setting both the `flex-direction` and `flex-wrap` properties.

Example

```
.flex-container {  
  display: flex;  
  flex-flow: row wrap;  
}
```

The CSS justify-content Property

The `justify-content` property is used to align the flex items when they do not use all available space on the main-axis (horizontally). The `justify-content` property can have one of the following values:

- `center`
- `flex-start`
- `flex-end`
- `space-around`
- `space-between`
- `space-evenly`

Example

The `center` value positions the flex items in the center of the container:

```
.flex-container {  
  display: flex;  
  justify-content: center;  
}
```


Example

The **flex-start** value positions the flex items at the beginning of the container (this is default):

```
.flex-container {  
  display: flex;  
  justify-content: flex-start;  
}
```

Example

The **flex-end** value positions the flex items at the end of the container:

```
.flex-container {  
  display: flex;  
  justify-content: flex-end;  
}
```

Example

The `space-between` value displays the flex items with space between them:

```
.flex-container {  
  display: flex;  
  justify-content: space-between;}
```

Example

The `space-around` value displays the flex items with space around them:

```
.flex-container {  
  display: flex;  
  justify-content: space-around;  
}
```

Example

The `space-evenly` value displays the flex items with equal space around them:

```
.flex-container {  
  display: flex;  
  justify-content: space-evenly;  
}
```

The CSS align-items Property

The `align-items` property is used to align the flex items when they do not use all available space on the cross-axis (vertically). The `align-items` property can have one of the following values:

- `center`
- `flex-start`
- `flex-end`
- `stretch`
- `baseline`
- `normal`

Example

The `center` value positions the flex items in the middle of the container:

```
.flex-container {  
  display: flex;  
  height: 200px;  
  align-items: center;  
}
```

Please practice all other properties.

- `flex-start`
- `flex-end`
- `stretch`
- `baseline`
- `normal`

The CSS align-content Property

The `align-content` property is used to align the flex lines.

The `align-content` property is similar to `align-items`, but instead of aligning flex items, it aligns the flex lines.

The `align-content` property can have one of the following values:

- `center`
- `stretch`
- `flex-start`
- `flex-end`
- `space-around`
- `space-between`
- `space-evenly`

In the following examples we use a `600` pixels high container, with the `flex-wrap` property set to `wrap`, to better demonstrate the `align-content` property.

Example

With `center`, the flex lines are packed toward the center of the container:

```
.flex-container {  
  display: flex;  
  height: 600px;  
  flex-wrap: wrap;  
  align-content: center;  
}
```

- `stretch`
- `flex-start`
- `flex-end`
- `space-around`
- `space-between`
- `space-evenly`

Perfect Centering

In the following example we will solve a common style problem: perfect centering.

SOLUTION

Set both the `justify-content` and `align-items` properties to `center`, and the flex item will be perfectly centered:

Example

```
.flex-container {  
  display: flex;  
  height: 300px;  
  justify-content: center;  
  align-items: center;  
}
```

The CSS Flex Items

The direct child elements of a flex container automatically becomes flex items.

Example

```
<div class="flex-container">  
  <div>1</div>  
  <div>2</div>  
  <div>3</div>  
  <div>4</div>  
</div>
```

The order Property

The **order** property specifies the order of the flex items inside the flex container. The first flex item in the code does not have to appear as the first item in the layout. The order value must be a number, default value is 0.

The CSS properties we use for flex items are:

- order
- flex-grow
- flex-shrink
- flex-basis
- flex
- align-self

Example

The *order* property can change the order of the flex items:

```
<div class="flex-container">  
  <div style="order: 3">1</div>  
  <div style="order: 2">2</div>  
  <div style="order: 4">3</div>  
  <div style="order: 1">4</div>  
</div>
```

The flex-grow Property

The **flex-grow** property specifies how much a flex item will grow relative to the rest of the flex items.

The value must be a number, **default value is 0**.

Example

Make the third flex item grow eight times faster than the other flex items:

```
<div class="flex-container">  
  <div style="flex-grow: 1">1</div>  
  <div style="flex-grow: 1">2</div>  
  <div style="flex-grow: 8">3</div>
```

The flex-shrink Property

The **flex-shrink** property specifies how much a flex item will shrink relative to the rest of the flex items.

The value must be a number, **default value is 1.**

Example

Do not let the third flex item shrink as much as the other flex items:

```
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div style="flex-shrink: 0">3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
  <div>7</div>
  <div>8</div>
  <div>9</div>
  <div>10</div>
</div>
```

The flex-shrink Property

The **flex-shrink** property specifies how much a flex item will shrink relative to the rest of the flex items.

The value must be a number, **default value is 1**.

The align-self Property

The **align-self** property specifies the alignment for the selected item inside the flexible container.

The **align-self** property overrides the default alignment set by the container's **align-items** property.

Example

Align the third flex item in the middle of the container:

```
<div class="flex-container">  
  <div>1</div>  
  <div>2</div>  
  <div style="align-self: center">3</div>  
  <div>4</div>  
</div>
```

Flex Container Properties

- display
- flex-direction
- flex-wrap
- flex-flow Combination of flex-direction and flex-wrap
- justify-content Defines the direction of items among the main axis
- align-items Defines the layout of items among the cross axis
- align-content Similar to the justify-content and its useful when there are a lot of rows in the container

Practicing session

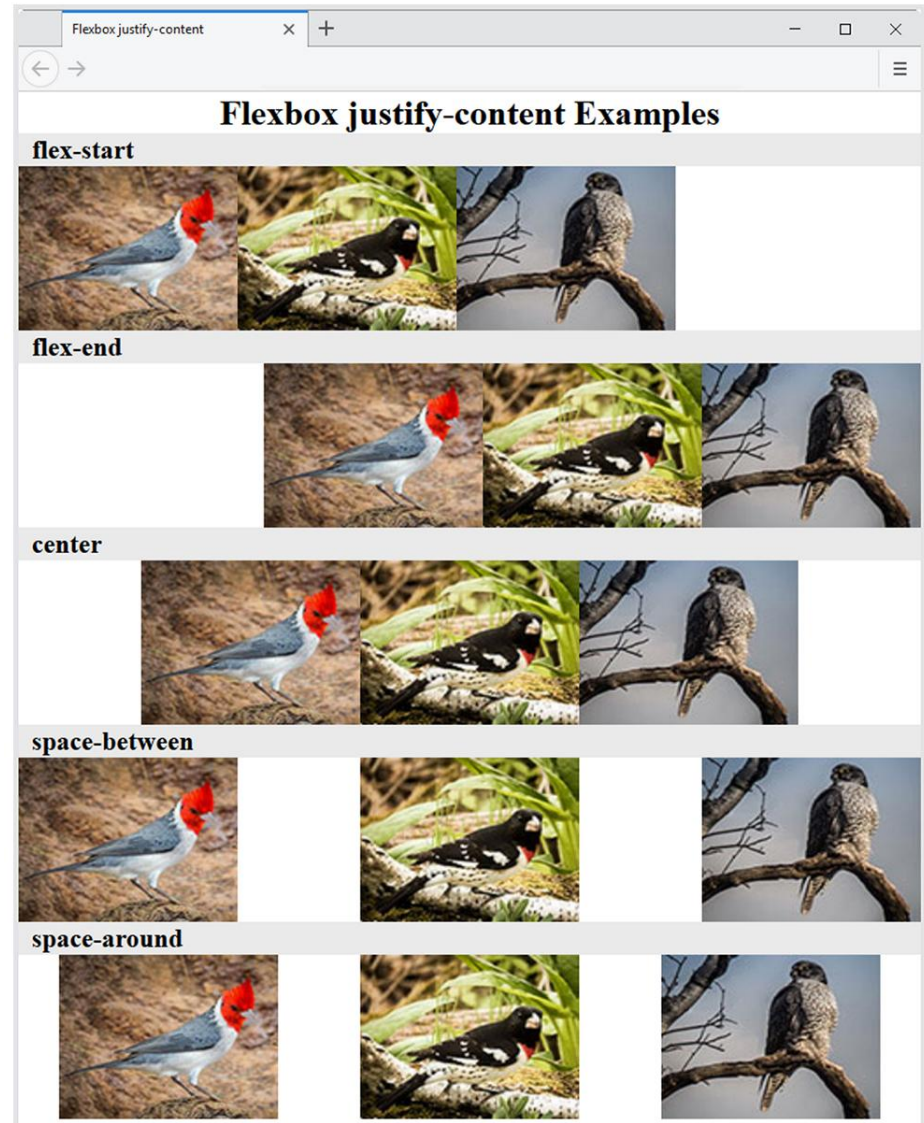
- **Flex Container Properties**
- `Display:flex;` but unnecessary space will appear
- `Display:inline-flex;` unnecessary space will be removed
- `Flex-direction:row;` or `Flex-direction:row-reverse;`
- `Flex-direction:column;` or `Flex-direction:column-reverse;`
- `Flex-wrap:Nowrap;` `Flex-wrap:wrap;`
- `Flex-wrap:wrap-reverse;` or `Flex-flow:row wrap-reverse;`
- `Justify-content:flex-start;` or `flex-end;` `center;` `space-between;` `space-around;` `space-evenly;`
- `Align-items:stretch;` `flex-start;` `flex-end;` `center;`
- **Flex ItemProperties---**
- `Order: 1;` etc.
- `Flex-grow:1;` by default its 0;
- `Flex-shrink:4;`
- `Align-self:flex-start;` `flex-end;` `center;`

The justify-content Property

Configures how the extra space along the main axis should be displayed

The justify-content property is used to align the flex items

when they do not use all available space on the main-axis (**horizontally**).

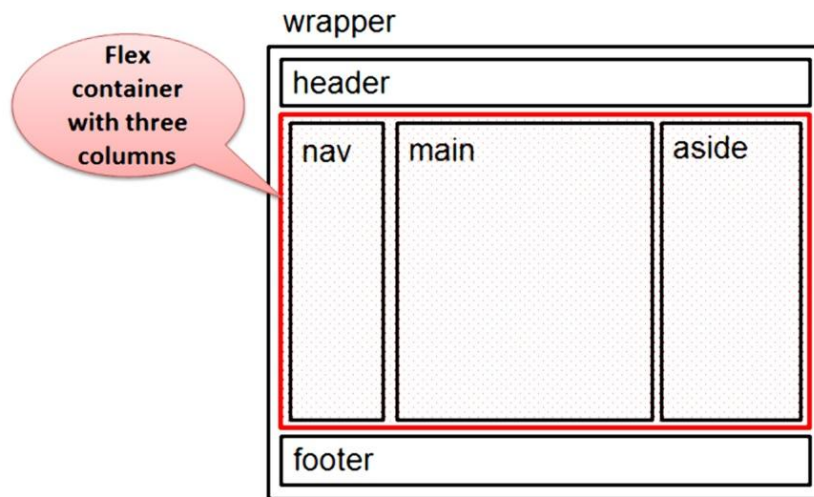


More flexbox Properties

- The align-items Property
 - Configures how the browser displays extra space along the cross-axis
- The flex-flow Property
 - Shorthand to configure flex-direction and flex-wrap properties
- **The order Property**
 - Causes the browser to display flex items in different order than they are coded
 - Warning – this could be an accessibility issue

Configure Flex Items

- By default flex items are flexible in size and allocated the same amount of space in the flex container
- The flex property
 - Customizes the size of each flex item
 - Indicated the flex grow factor
 - Indicates the flex shrink factor
 - Can be used to indicate a proportional flexible item



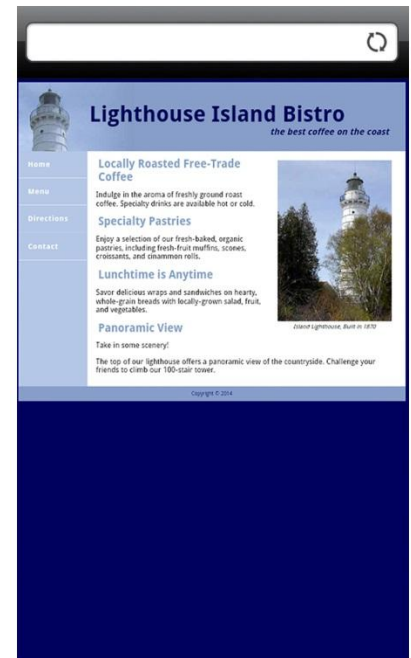
```
nav { flex: 1; }  
main { flex: 7; }  
aside { flex: 2; }
```


Viewport Meta Tag

Default action for most mobile devices is to zoom out and scale the web page

- Viewport Meta Tag
 - Created as an Apple extension to configure display on mobile devices
 - Configures width and initial scale of browser viewport

```
<meta name="viewport"
content="width=device-width, initial-scale=1.0">
```



CSS Debugging Tips

- Manually check syntax errors
- Use W3C CSS Validator to check syntax errors
 - <http://jigsaw.w3.org/css-validator/>
- Configure temporary background colors
- Configure temporary borders
- Use CSS comments to find the unexpected
`/* the browser ignores this code */`
- Don't expect your pages to look exactly the same in all browsers!
- Be patient!

Summary

- This chapter expanded your CSS and HTML skillset.
- You configured web pages with CSS Flexible Box Layout (flexbox).
- You configured web pages for mobile display using the viewport meta tag.

Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.